Homework – Web Frontend Developer

Summary

Build a real-time chat application that supports multiple participants. This exercise is designed to show us how you think about architecture, write code, and make trade-offs — not to produce a production-ready product.

Architecture

You may choose **either** approach:

Option A: Client-Server

Traditional architecture with a server relaying messages between clients. Use any real-time transport you're comfortable with (WebSocket, SSE, polling, etc.).

Option B: Peer-to-Peer (preferred)

Use **WebRTC DataChannels** for direct messaging between peers. A lightweight signaling server is expected and can be minimal.

> Choosing Option B demonstrates familiarity with real-time communication patterns and will be evaluated favorably — but a well-designed Option A solution is absolutely acceptable. We value good architecture over technology choice.

Regardless of which option you choose, **document your message protocol** (format, event types, how edits/deletes are represented) in your README or in the code itself.

Requirements

Core (Must Have)

1. **Real-time messaging** – messages appear on all connected clients without refresh
2. **Multi-participant support** – at least 2 clients connected simultaneously
3. **Presence** – display a list of who is currently in the session
4. **Edit own messages** – other participants see that the message was edited
5. **Delete own messages** – other participants see that the message was deleted
6. **Styled UI** – match the provided design mockups

Technical Constraints

- **Frontend:** TypeScript + React (preferred), or a framework you're comfortable with
- **Server/signaling:** any language
- Use a **git repository** with frequent, meaningful commits
- Include a **README** with:
  - How to build and run the application
  - Which architecture option you chose, and why
  - Brief description of your message protocol
  - Time spent
  - Trade-offs, known issues, and what you'd improve with more time
- **Do not** use "Pexip" anywhere in the repository (name, README, metadata, etc.)

Quality & Testing

- Include **automated tests** for at least the core message logic (unit and/or integration)
- Demonstrate a clear separation of concerns and testable architecture
- Briefly describe your testing strategy in the README

Edge Cases to Consider

- What happens when a participant disconnects and reconnects?
- What happens when a new participant joins — do they see message history?
- How does the UI handle a large volume of messages?

You don't have to solve all of these, but we'd like to see you **acknowledge and discuss** the ones you didn't address.

Bonus Tasks

Pick any that interest you — these are additions to the core requirements, not replacements.

Area	Task
Media	Image/file sharing with preview
Real-time	Typing indicators, read receipts
Performance	Virtualized message list handling 10k+ messages
Security	End-to-end encryption (only clients can read messages)
DevOps	Dockerfile / docker-compose for one-command startup
Accessibility	Keyboard navigation, screen reader support
Testing	E2E tests (e.g. Playwright, Cypress)
API Design	REST endpoint for message history with pagination
Rich Content	Link previews, emoji picker, Giphy integration
Theming	Alternative layouts or themes, design-system approach
AI	Smart features (summarization, auto-complete, moderation)

What We Value

We care about **how you think and work**, not just the end result. We're looking at your code the way we'd review a pull request from a colleague — is it clean, thoughtful, and well-communicated? In the follow-up interview, we'll walk through your code together to discuss your approach, decisions, and trade-offs.

Hints

- Test with **2+ browser windows** or devices — this is a multi-participant app
- Test with a **large volume of messages** to see how your UI holds up
- The design is narrow, but this is **not meant to be mobile-only**
- **KISS** — Keep It Simple. A well-structured minimal solution beats a messy feature-rich one

---

Time Guidance

Aim for **6–8 hours**. We understand people work at different rates and have varying availability — there is no deadline. Just come back to us when you've found time to look at this.

We are **not** expecting a production-quality solution. We would rather see you demonstrate good software design principles, clear thinking, and honest documentation of trade-offs than a fully polished implementation.

> However, as there may be other candidates in this process, we may not be able to hold the position open indefinitely.

If anything is unclear or you have questions, please reach out — we're happy to help.